什么是 PAN、LAN、MAN、WAN？它们的覆盖范围分别是多少？
Personal Area Network (PAN): about 1–10 meters, typically within a room.
Local Area Network (LAN): about 10 meters to 1 km, such as a building or campus.
Metropolitan Area Network (MAN): about 10–100 km, covering a town or city.
Wide Area Network (WAN): about 1000 km, spanning countries or continents.
The Internet: roughly 10,000 km, global scale.

它们的大小关系应该如何理解？ | PAN < LAN < MAN < WAN.
A WAN is absolutely larger than a MAN; reversing this relationship is incorrect.

什么是网络协议（Protocol）？
Protocols define the format and order of messages sent and received among network entities, and the actions taken upon message transmission and receipt.

五大 RIR 分别是哪些？
ARIN — American Registry for Internet Numbers
RIPE NCC — Réseaux IP Européens Network Coordination Centre
APNIC — Asia-Pacific Network Information Centre
LACNIC — Latin America and Caribbean Network Information Centre
AFRINIC — African Network Information Centre

ICANN和IANA关系 | ICANN sets up the overall system and delegates operational responsibilities to IANA.

IANA 的主要职责是什么？ | IANA operates at the highest international level and coordinates global IP-related resources. | It further delegates work to five Regional Internet Registries (RIRs).

（Network Edge）？ | The network edge consists of hosts, including clients and servers. Servers are often located in data centers and connected via national or global ISPs.

（Network Core）？ | The network core is a set of interconnected routers forming a network of networks.

（Access Network）？ | The access network physically connects an end system to the first edge router. Examples include home networks, content provider networks, and datacenter networks.

Packet Switching :Data is sent in discrete packets through shared network resources.
Circuit Switching : A dedicated circuit is reserved per call, such as in the traditional telephone network.

Packet switching shares resources with variable delay. | Circuit switching reserves a dedicated path with fixed bandwidth and predictable delay.| **为什么分组交换更高效？** : Packet switching is efficient for bursty data, supports resource sharing, and requires no call setup. However, congestion may cause delay and packet loss, so protocols are needed for reliability and congestion control. | **如何让分组交换表现得像电路交换？** : Complex mechanisms attempt to make packet switching more circuit-like.

Application	Protocol	Transport
file transfer / download	**FTP** (RFC 959)	**TCP**
e-mail	**SMTP** (RFC 5321)	**TCP**
Web documents	**HTTP/1.1** (RFC 7320)	**TCP**
Internet telephony	**SIP** (RFC 3261), **RTP** (RFC 3550), proprietary	**TCP or UDP**
streaming audio/video	**HTTP** (RFC 7320), **DASH**	**TCP**
interactive games	WoW, FPS (proprietary)	**UDP or TCP**

HTTP/2协议 : Key goal: **decreased delay** in multi-object HTTP requests
HTTP/2 (RFC 7540, 2015) increases **server flexibility** in sending objects to clients:
- Methods, status codes, and most header fields unchanged from HTTP/1.1
- Transmission order of requested objects based on **client-specified priority** (not necessarily FCFS)
- Server can **push unrequested objects** to the client
- Objects divided into frames; frames scheduled to **mitigate head-of-line (HOL) blocking**
- HTTP/2 introduces a transport-level HOL blocking problem

Domain Name System (DNS) : A distributed database implemented as a hierarchy of many na servers | An **application-layer protocol** used by hosts and DNS servers for **name resolution** (address/name translation) | A core Internet function implemented at the application layer, with complexity at the network edge

DNS services and structure | **Hostname-to-IP-address translation** (used by HTTP, SMTP); **Host aliasing** (canonical: relay1.west-coast.enterprise.com, alias: enterprise.com); **Mail server aliasing**; **Load distribution** (replicated Web servers, multiple IPs per name) | **Why not centralize DNS**: **single point of failure**; **traffic volume**; **distant centralized database**; **maintenance overhead** | **Answer**: **does not scale** | **Scale examples**: **Comcast DNS ~600 billion queries/day**; **Akamai DNS ~2.2 trillion queries/day**

Content Distribution Networks (CDNs) |Through the use of CDNs, Web caches are increasingly playing an important role in the Internet. A CDN company installs many **geographically distributed caches** throughout the Internet, thereby localizing much of the traffic. There are **shared CDNs** (such as Akamai and Limelight) and **dedicated CDNs** (such as Google and Netflix).

Peer-to-peer (P2P) architecture :**no always-on server** | arbitrary end systems directly communicate | peers request service from other peers, provide service in return to other peers | **self scalability** – new peers bring new service capacity, and new service demands | peers are intermittently connected and change IP addresses | examples: P2P file sharing (BitTorrent), streaming, VoIP (earlier Skype)

Client–Server (C/S) : **Distribution time increases linearly and without bound as the number of peers increases**; servers are the main distributors of data | **Peer-to-Peer (P2P)** : **Self-scaling architecture**; peers act as both **consumers and redistributors** of data, enabling better scalability

client–server Server : always-on host | permanent IP address | data centers server farms scaling。

client : contact, communicate with server | intermittently connected | dynamic IP address

SMTP RFC 5321 | **U**ses **TCP** for reliable email transfer from sending mail server (client) to receiving server on **port 25** | **D**irect transfer from sending server to receiving server | **T**hree phases: SMTP handshaking (greeting), SMTP message transfer, SMTP closure | **C**ommand/response interaction (like HTTP): commands in ASCII text, responses with status code and phrase

OTT: "**over the top**" challenges: **coping with a congested Internet from the "edge"** | what content to place in which CDN node? | from which CDN node to retrieve content? At which rate?

Socket 编程介绍-Socket Programming | **Two socket types**: **UDP** (unreliable datagram) and **TCP** (reliable byte-stream oriented) | **UDP application example**: client sends data from keyboard → server converts to uppercase → server replies → client displays result | **UDP socket programming**: **no connection**, **no handshaking**, sender **explicitly includes destination IP and port**, receiver **extracts sender IP and port**, data may be **lost or out of order**, provides **unreliable datagram transfer** | **TCP socket programming**: **client must contact server**, server must be running and listening, client **creates TCP socket** specifying server IP and port, TCP **connection established**, server creates a **new socket per client**, supports **multiple clients**, uses **source port numbers to distinguish clients**, provides **reliable, in-order byte-stream ("pipe") transfer**

Network service model |What service model for chanel transporting datagrams from sender to receiver?

Ethernet： "Dominant" wired LAN technology: First widely used LAN technology | Simple, cheap | Single chip, multiple speeds |Kept up with speed race: 10 Mbps – 400 Gbps

什么是 Internet Society（ISOC）？
Internet Society (ISOC) is a global non-profit organization that supports the open development, evolution and use of the Internet.
It provides organizational support for bodies such as the IAB, IETF, and IRTF.

Internetworking Principles 基于哪些核心原则?

Minimalism and autonomy : no internal changes required to interconnect networ
Best-effort service model. | Stateless routing. | Decentralized control.
These principles define today's Internet architecture.

Network Architecture	Service Model	Bandwidth	Loss	Order	Timing
Internet	**best effort**	**none**	**no**	**no**	**no**
ATM	**Constant Bit Rate**	**constant rate**	**yes**	**yes**	**yes**
ATM	**Available Bit Rate**	**guaranteed min**	**no**	**yes**	**no**
Internet	**Intserv Guaranteed** (RFC 1633, 1994)	**yes**	**yes**	**yes**	**yes**
Internet	**Diffserv** (RFC 2475, 1998)	**possible**	**possibly**	**possibly**	**no**

Internet 使用哪五层协议栈？ | **Application**: supporting network applications : HTTP, IMAP, SMTP, DNS | **Transport**: process-to-process data transfer : TCP, UDP | **Network**: routing of datagrams from source to destination : IP, routing protocols | **Link**: data transfer between neighboring network elements : Ethernet, 802.11 (WiFi), PPP | **Physical**: bits "on the wire" | **Two fundamental approaches to moving data through a network of links and switches**

DNS 域名解析：迭代查询（Iterated Query） | **流程**: requesting host → **local DNS** → **root DNS** → local DNS → **TLD DNS** → local DNS → **authoritative DNS** → local DNS → requesting host | **目标**: **gaia.cs.umass.edu** | **含义**: **被查询服务器只返回下一步应联系的服务器**（"我不知道，但你去问它"）

DNS 域名解析：递归查询（Recursive Query） | **流程**: host → **local DNS** → **root DNS** → **TLD DNS** → **authoritative DNS** → TLD DNS → root DNS → local DNS → host | **目标**: **gaia.cs.umass.edu** | **特点**: **由被查询服务器完成解析，上层 DNS 负载较重**

Routing: 性质：Control Plane的功能 | 动作：决定分组从源到目的地的路径。 |范围：**全局 (Network-wide)，涉及所有路由器。**

Forwarding: 性质：Data Plane的功能 | 动作：将分组（Packet）从路由器的输入链路移动到输出链路。 | 范围：**本地 (Local)，仅关注单个路由器内部。**

节点时延：dnodal = dproc + dqueue + dtrans + dprop；**端到端时延**：dend-end = N × (dproc + dtrans + dprop)（通常不计 dqueue）；**处理时延 dproc**：节点处理分组所需时间；**排队时延 dqueue**：分组在队列中等待时间，取决于拥塞；**传输时延 dtrans** = L / R（L 比特，R 比特/秒）；**传播时延 dprop** = D / S（D 米，S 米/秒）。
例1（dnodal）：设 dproc=1 ms，dqueue=2 ms，L=12000 bit，R=10 Mb/s ⇒ dtrans=12000/10^7=1.2 ms，D=100 km，S=2×10^8 m/s ⇒ dprop=0.5 ms，则 dnodal=1+2+1.2+0.5=4.7 ms。
例2（dend-end）：N=5，dproc=1 ms，L=12000 bit，R=10 Mb/s ⇒ dtrans=1.2 ms，D=100 km，S=2×10^8 m/s ⇒ dprop=0.5 ms，则 dend-end=5×(1+1.2+0.5)=13.5 ms。

Streaming multimedia: DASH : Client "intelligence": **when** to request chunk (avoid buffer starvation/overflow) | **what** encoding rate to request (higher quality if bandwidth allows) **where** to request chunk (server close to client or high bandwidth) :

Streaming video = encoding + DASH + playout buffering

IAB、IETF、IRTF、IESG、IRSG 分别是什么？
IAB: Internet Architecture Board
IETF: Internet Engineering Task Force
IRTF: Internet Research Task Force
IESG: Internet Engineering Steering Group
IRSG: Internet Research Steering Group

$$[1\ \text{Gb/s link, 35 users, each active at 100 Mb/s with } p = 0.1;\ \max \text{ active users} = 10;\ X \sim \text{Binomial}(35, 0.1);$$
$$P(X > 10) = \sum_{k=11}^{35} \binom{35}{k} (0.1)^k (0.9)^{35-k} \approx 4.2 \times 10^{-4}.]$$
$$[\binom{35}{k} = \frac{35!}{k!(35-k)!},\quad \text{表示从 35 个用户中选出 } k \text{ 个的组合数。}]$$

HTTP 是什么 : Hypertext Transfer Protocol | Web's application-layer protocol
Client/server model: **Client**: browser that requests, receives (using HTTP), and displays Web objects | **Server**: Web server sends objects in response to requests using HTTP

HTTP uses TCP: Client initiates **TCP connection** (creates socket) to server, **port 80** | Server accepts TCP connection from client | HTTP messages exchanged between browser (HTTP client) and Web server (server) | TCP connection closed

HTTP is "stateless" : **Server maintains no information about past client requests** | Protocols tha maintain "state" are complex | Past history (state) must be maintained |If server/client crashes, state may be inconsistent and must be reconciled

HTTP/2 到 HTTP/3 : HTTP/2 over a single TCP connection means:**Packet loss recovery still stalls all object transmissions** | As in HTTP/1.1, browsers open **multiple parallel TCP connections** to reduc stalling and increase throughput | **No security** over a vanilla TCP connection

HTTP/3:Adds **security** and **per-object error and congestion control over UDP** | Enables **more pipelining** | Specifically designed to **eliminate HOL blocking**

HTTP/1.1: **HOL blocking** | HTTP 1.1: client requests 1 large object (e.g., video file) and 3 smaller objects

HTTP/2: **mitigating HOL blocking** |HTTP/2: objects divided into **frames**, **frame transmission interleav**

UDP 核心特性 (Core Characteristics)

"no frills," "bare bones" Internet transport protocol

"best effort" service**, UDP segments may be: lost | delivered out-of-order to app

connectionless: no handshaking between UDP sender, receiver | each UDP segment handled independently of others

Why is there a UDP? |no connection establishment (which can add RTT delay) | simple: no connection state at sender, receiver | small header size | no congestion control: UDP can blast away as fast as desired! | can function in the face of congestion

TCP service : **Reliable transport** between sending and receiving processes | **Flow control**: sender does not overwhelm receiver | **Congestion control**: throttles sender when network is overloaded **Connection-oriented**: setup required between client and server processes | **Does not provide**: timing guarantee minimum throughput guarantees, security

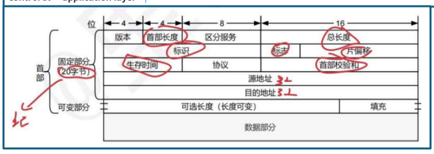
UDP service : **Unreliable data transfer** between sending and receiving processes | **Does not provi** reliability, flow control, congestion control, timing, throughput guarantees, security, or connection set

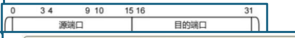
possible services for individual datagrams: ▪ **guaranteed delivery** ▪ **guaranteed delivery with bounded delay** (e.g., less than 40 msec) | possible services for a flow of datagrams: ▪ **in-order datagram delivery** ▪ **guaranteed minimum bandwidth** to flow ▪ **security** | This is only a partial list of services that a networ layer could provide — there are countless variations possible.

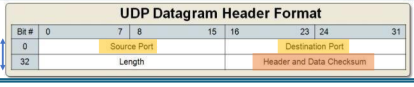
UDP 校验和（Checksum） | **Goal**: detect errors (i.e., flipped bits) in transmitted segment | **Sender**: treat contents of UDP segment (including UDP header fields and IP addresses) as sequence of **16-bit integers**; checksum = **one's complement sum** of segment content; checksum value put into **UDP checksum field** | **Receiver**: compute checksum of received segment; compare with checksum field (**NO** → error detected, **YES** → no error detected, but errors may still exist) | **Internet checksum example**: carryout from **MSB** added back; sender performs **1's complement**; receiver adds all 16-bit words including checksum; no error ⇒ result = **1111111111111111**; any 0 bit ⇒ error | **Weak protection**: bit flips may not change checksum

UDP 多路分解（Demultiplexing） | **Connectionless demultiplexing** | Host uses **IP address + port number** to direct segment to socket | Sender specifies **destination IP address** and **destination port #** Receiver checks **destination port #** and delivers to matching socket | Same dest IP + dest port but different source IP/port ⇒ delivered to **same socket** | **Source Port** provides **return address**

UDP 常见应用（Common Applications） | **Streaming multimedia apps** (loss tolerant, rate sensitive) | **DNS** | **SNMP** | **HTTP/3** | If reliability needed over UDP (e.g., **HTTP/3**): add **reliability** and **congestion control** at **application layer**





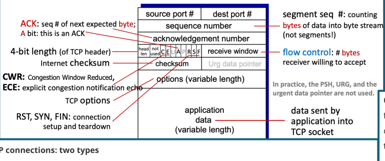


UDP Datagram Header Format

Bit #	0–7, 8–15	16–23, 24–31
0	Source Port	Destination Port
32	Length	Header and Data Checksum

8 Byte

Data plane | **local, per-router function** | determines how a datagram arriving on a router **input port** is forwarded to a router **output port** | based on **values in the arriving packet header**



32 bits
source port #
dest port #
sequence number
acknowledgement number
head len
not used
C E U A P R S F
receive window
checksum
Urg data pointer
options (variable length)
application data (variable length)
ACK: seq # of next expected byte; A bit: this is an ACK
4-bit length (of TCP header)
Internet checksum
CWR: Congestion Window Reduced,
ECE: explicit congestion notification echo
TCP options
RST, SYN, FIN: connection setup and teardown
segment seq #: counting bytes of data into byte stream (not segments!)
flow control: # bytes receiver willing to accept
In practice, the PSH, URG, and the urgent data pointer are not used.
data sent by application into TCP socket

HTTP connections: two types

Default mode of HTTP/1.0 — **Non-persistent HTTP** : TCP connection opened | At most **one object sent** over TCP connection | TCP connection closed

Downloading multiple objects requires **multiple connections**

Default mode of HTTP/1.1+ — **Persistent HTTP** :TCP connection opened to a server | **Multiple objects sent over a single TCP connection** between client and server | TCP connection closed

HTTP状态码 200 OK：请求成功，请求的对象在本消息中 | **301 Moved Permanently**：请求的对象已永久移动，新位置在本消息的 **Location** 字段中 | **400 Bad Request**：服务器无法理解请求消息 | **404 Not Found**：服务器上未找到请求的文档 | **505 HTTP Version Not Supported**：服务器不支持请求中使用的 HTTP 版本

Aspect	Link State (LS)	Distance Vector (DV)
Message Complexity	**High** broadcasting entire topology, O(nE)	**Low** Exchange information between neighbors
Convergence Speed	**Fast** ($O(n^2)$ algorithm), may oscillate	**Slow**
Robustness	**Good** (Nodes compute their own tables; errors usually affect only locally)	**Poor**
Typical Protocols	OSPF, IS-IS	RIP, BGP

3. 迁移技术 (Transition)
- 隧道 (Tunneling)：
  - 场景：两个 IPv6 岛屿中间隔着 IPv4 的海洋。
  - 做法：把整个 IPv6 数据报作为载荷 (Payload) 封装在 IPv4 数据报中。
  - IPv4 头部中的 Protocol 字段设为 41（表示载荷是 IPv6）。

3.6 IPv6 (PPT 4-96 ~ 4-105)

1. 动机: 32 位 IPv4 地址耗尽。
2. 头部变化 (Header Changes) —— 对比考点
   - 地址：扩充到 128 位。
   - **固定头部：40 字节**（IPv4 是可变的 20+）。
   - No Fragmentation：路由器**不再分片**。如果包太大，直接丢弃并回送 ICMP "Packet Too Big"。
   - No Checksum：去掉了校验和，为了提高路由器处理速度（以此为代价）。
   - Next Header：类似于 IPv4 的 Protocol 字段，但也用于指向扩展头部（Extension Headers）。

ICMP: Internet Control Message Protocol | Used by hosts and routers to communicate **network-level information** | **Error reporting**: unreachable host, network, port, or protocol | **Echo request/reply** (used by **ping**) | Network-layer protocol **"above" IP** | ICMP messages carried inside **IP datagrams** | ICMP message format: **type**, **code**, plus **first 8 bytes of the IP datagram** that caused the error

Routing Algorithm Classification | **By information source**: centralized (link-state) vs decentralized (distance-vector) | **Centralized / Global**: all routers have complete topology and link cost information | **Distributed / Decentralized**: routers know only neighbors and link costs, exchange info iteratively

Link-State algorithm (Dijkstra's Algorithm) | **Basic idea**: each router learns full network topology and link costs | **Information dissemination**: link-state broadcast to all nodes | **Computation**: each node independently computes shortest paths to all other nodes | **Result**: routing table with least-cost paths from the node to every destination

Characteristics of Intra-AS Routing (IGP) | **Scope**: routing within same AS ("network") | **Uniformity**: all routers within a single AS must run the **same intra-domain protocol** | **Primary Function**: determine routing entries for destinations **within its own AS** | **Protocols**: **RIP** (Routing Information Protocol), **EIGRP** (Enhanced Interior Gateway Routing Protocol), **OSPF** (Open Shortest Path First), **IS-IS**

Characteristics of Inter-AS Routing (EGP) | **Scope**: routing among AS'es | **Execution Point**: performed by **gateway routers** at the **edge of an AS** | **Primary Function**: manage routing between domains — **learn reachability** of external destinations via neighboring ASes and **propagate information** to all routers inside its own AS | **Goal**: works with Intra-AS routing to determine forwarding table entries for **external destinations**

Per-router control plane - 架构：Distributed - 原理：

每个路由器都有一个路由选择算法组件。它们之间互相交

换路由信息，各自计算出自己的转发表。 - 关键点：大脑分散在每个路由器里。

SDN - Software-Defined Networking - 架构：Logically Centralized - 原理：路由器里的"大脑"被拿走了，
下转发功能。一个Remote Controller负责计算所有路由器的转发表，然后Install到路由器中 。 关键点：大脑
端的服务器里，路由器只负责执行。

DHCP 概述（DHCP overview） | Host broadcasts **DHCP Discover** message (optional) | DHCP server responds with **DHCP Offer** message (optional) | Host requests IP address with **DHCP Request** message DHCP server assigns address with **DHCP ACK** message

ARP：Address Resolution Protocol | ARP allows machines to resolve **IP-to-MAC address mappings without maintaining a permanent binding record | To avoid a centralized database, ARP uses a **low-level protocol** to **dynamically bind addresses**

TCP 核心特性（Core Characteristics） | **point-to-point**: one sender, one receiver | **reliable, in-order byte stream**: no "message boundaries" | **pipelined**: TCP congestion and flow controls set window size | **send & receive buffers** | **full duplex data**: bi-directional data flow in same connection; **MSS**: maximum segment size | **connection-oriented**: handshaking (exchange of control msgs) initializes sender and receiver state before data exchange | **flow controlled**: sender will not overwhelm receiver

序列号与确认号（Sequence Numbers and ACKs） | **Sequence numbers**: byte stream "number" of the first byte in segment's data | **Acknowledgements**: seq # of next byte expected from other side; **cumulative ACK** | **Q**: how receiver handles out-of-order segments? | **A**: TCP spec doesn't say, up to implementor (drop or buffer)

可靠数据传输（Reliable Data Transfer, RDT） | TCP creates rdt service on top of IP's unreliable service | **Retransmissions triggered by**: **timeout events**; **duplicate ACKs** | **TCP fast retransmit**: on **3 duplicate ACKs**, presume loss and **retransmit immediately** without waiting for timeout

流量控制（Flow Control） | **Goal**: receiver controls sender to avoid buffer overflow | **Mechanism**: receiver advertises free buffer via **receive window (rwnd)** in TCP header; sender limits **in-flight data ≤ rwnd**; guarantees no receive-buffer overflow

连接管理（Connection Management） | **Three-Way Handshake**: **SYN** (client sends SYN, random client_isn) → **SYN-ACK** (server sends SYN+ACK, ack=client_isn+1, random server_isn) → **ACK** (client sends ACK, ack=server_isn+1) | **Closing a Connection**: **FIN** (client) → **ACK** (server) → **FIN** (server) → **ACK** (client, enters **TIME_WAIT**) → server receives ACK, connection closed

拥塞控制（Congestion Control） | **Goal**: prevent one TCP connection from swamping the network | **Approach**: sender probes bandwidth by increasing window until loss | **AIMD**: **additive increase** (+1 MSS per RTT), **multiplicative decrease** (halve CongWin) | **Stages**: **Slow Start** (CongWin=1 MSS, doubles each RTT until **ssthresh**), **Congestion Avoidance** (linear +1 MSS/RTT), **Detection & Reaction** (timeout → ssthresh=CongWin/2, CongWin=1 MSS; **3 dup ACKs** → ssthresh=CongWin/2, CongWin=CongWin/2, **Fast Recovery**) | **TCP CUBIC**: cubic window growth; fast growth after loss, slows near previous **W_max**

TCP 多路分解（Demultiplexing） | **Connection-oriented demultiplexing** | TCP socket identified by **4-tuple** {source IP, source port, dest IP, dest port} | receiver uses **4-tuple** to demux | different source IP/port → **different sockets** | web servers maintain **separate sockets per client**

Router architecture High-level view of a generic router architecture:
- **Routing processor** : **Performs routing and management** (**control-plane functions**) | Control plane (**software**), operates in millisecond time frame
- **Forwarding data plane** : **Implemented in hardware**, operates in nanosecond time frame | **Avoids centralized processing bottleneck**
- **Switching fabric** : **High-speed internal network** connecting input ports to output ports
- **Shadow copy** : **Forwarding table copied** from routing processor to line cards over a separate bus

Consequence of HOL blocking： The switch's **throughput is dramatically reduced**, preventing the switch from efficiently using all of its available output ports.
Head-of-the-Line (HOL) blocking: a **queued datagram at the front of a queue prevents ot datagrams** in the queue from moving forward.

Packet Scheduling Disciplines | **FIFO (First-In First-Out)**: packets are transmitted in the order they arrive | simple to implement | no traffic differentiation | can lead to high delay and unfairness when congestion occurs

Priority Queuing (PQ) | packets are classified into priority classes | higher-priority packets are always served first | **Non-preemptive**: once transmission of a packet starts, it is not interrupted | lower-priority traffic may suffer starvation

Weighted Fair Queuing (WFQ) | generalized form of fair queuing | each flow is assigned a weight | bandwidth is shared proportionally to weights | provides fairness and bandwidth guarantees

Round Robin (RR) | packets (or flows) are served in turn | simple and prevents starvation | does not consider packet size differences | may still be unfair when packet sizes vary

协议栈中数据的传输路径是怎样的？
Source path: application → transport → network → link → physica
Destination path: physical → link → network → transport → applic

Three Types of Services Performed by Middleboxes | **NAT Translation**: NAT boxes implement private network addressing; rewrite IP addresses and port numbers in datagram headers | **Security Services**: firewalls block traffic based on header-field values; redirect packets for additional processing such as **Deep Packet Inspection (DPI)** and **Intrusion Detection Systems (IDS)** | **Performance Enhancement**: compression, content caching, and load balancing of service requests (e.g., HTTP request or search query) to one of a set of servers

CSMA (Carrier Sense Multiple Access) | **Simple CSMA**: listen before transmit | If channel is **sensed idle**, transmit entire frame | If channel is **sensed busy**, defer transmission | **Human analogy**: don't interrupt others | **CSMA/CD**: CSMA with **collision detection** | Collisions detected within a **short time** | **Colliding transmissions aborted**, reducing channel waste | Collision detection **easy in wired**, **difficult in wireless** | **Human analogy**: the polite conversationalist

802.11: Channels, association | A **WiFi jungle** is any physical location where a wireless station receives a sufficiently strong signal from **two or more APs** | **Spectrum divided into channels** at different frequencies | **AP admin chooses frequency** for AP | **Interference possible**: channel can be same as that chosen by neighboring AP | **Arriving host must associate with an AP** | Scans channels, listening for **beacon frames** containing AP's **SSID** and **MAC address** | **Selects AP** to associate with | May perform **authentication** | Then typically runs **DHCP** to get an **IP address in AP's subnet**

. Summary: How Does a Router Work Efficiently?
- 输入端口：收包 + 最长前缀匹配查表（硬件加速） → 决定输出口；
  Input port: Receive + longest-prefix match (hardware-accelerated) → decide output port;
- 交换结构：并行高速转发（Crossbar/多平面） → 避免内部瓶颈；
  Switching fabric: Parallel high-speed forwarding (Crossbar/multi-plane) → avoid internal bottlenecks
- 输出端口：排队 + WFQ等调度策略 → 按优先级/公平性发送；
  Output port: Queue + scheduling (e.g., WFQ) → send by priority/fairness;
- 路由处理器：后台计算路由 → 异步更新转发表 → 不干扰数据面；
  Routing processor: Compute routes in background → asynchronously update forwarding tables → n
- HOL阻塞？ → 用 VOQ 解决；
  HOL blocking? → Solved by VOQ;
- 输出拥塞？ → 用 WFQ + AQM 优化。
  Output congestion? → Optimized by WFQ + AQM.

Internet Architecture是什么？
Packet-switched datagram network.
IP is the glue at the network layer.
IP hourglass architecture: all hosts and routers run IP.
Stateless architecture: no per-flow state inside the network.

为什么需要SDN? / Why SDN?
- 网络管理困难 / Difficult network management
- 需要更容易的网络演进 / Need easier network evolution
- 打破厂商锁定 / Break vendor lock-in

IP fragmentation / reassembly
- Network links have an **MTU (Maximum Transmission Unit)**
  - **Largest possible link-layer frame**
  - Different link types have different MTUs
- Large IP datagrams may be **fragmented within the network**
  - **One datagram becomes several smaller datagrams**
  - **Reassembly occurs only at the destination**
  - IP header fields are used to identify and order fragments

SDN四个关键特征 / Four Key SDN Features

1. **流量控制的转发 / Flow-controlled forwarding**
   - 基于任意头部字段（不仅是目的IP）/ Based on any header fields (not
2. **控制与数据平面分离 / Separation of control and data planes**
3. **网络控制功能外置 / Network control functions external**
   - 控制器和应用在交换机外部 / Controller and apps external to switches
4. **可编程网络 / Programmable network**

Bad News Travels Slowly (Count-to-Infinity problem) | **Problem description**: when link cost increases or a link fails, incorrect low-cost routes may persist | **Effect**: distance estimates can increase slowly toward infinity | **Example scenario**: routers repeatedly increase path cost through each other

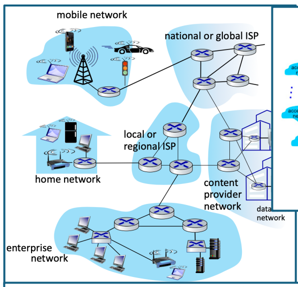
Poisoned Reverse | **Idea**: a router advertises an infinite distance to a destination on the link it uses to reach that destination | **Benefit**: helps prevent simple routing loops | **Limitation**: only partially solves the count-to-infinity problem

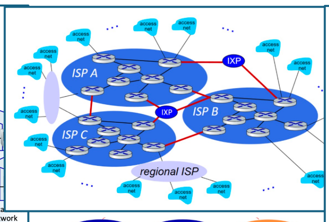
ARPANET 是什么时候提出的？ | In 1967, ARPANET was conceived by the Advanced Research Projects Agency (ARPA). | In 1969, the first ARPANET node became operational.

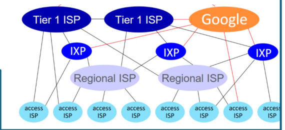
Network management includes the **deployment, integration,** and **coordination** of **hardware, software,** and **human elements** | to **monitor, test, poll, configure, analyze, evaluate,** and **control network and element resources** | in order to meet **real-time** and **operational performance** goals as well as **Quality of Service (QoS) requirements** | **at a reasonable cost**

Control plane | **network-wide logic** | determines how a datagram is **routed among routers** along the end-to-end path from **source host to destination host** | **two approaches: traditional routing algorithms** (implemented in routers) and **software-defined networking (SDN)** (implemented in remote servers)

OpenFlow abstraction | **Match + action**: a unifying abstraction for different network devices | **Router**: Match = longest destination IP prefix; Action = forward out a link | **Switch**: Match = destination MAC address; Action = forward or flood | **Firewall**: Match = IP addresses and TCP/UDP port numbers; Action = permit or deny | **NAT**: Match = IP address and port; Action = rewrite address and port







At "center": small # of well-connected large networks
- "tier-1" commercial ISPs (e.g., Level 3, Sprint, AT&T, NTT), national & internat
- content provider network (CDN) (e.g., Google, Facebook): private network its data centers to Internet, often bypassing tier-1, regional ISPs

Compare and contrast between packet switching and circuit switching | **Packet switching:** dat split into packets and sent independently, **no dedicated path**, **high efficiency**, but **variable delay/ji** (used by Internet/IP). **Circuit switching:** a **dedicated end-to-end circuit** is established first, **fixed bandwidth and stable delay**, but **low utilization when idle** (used by traditional telephone network

Present Satellite Communication Services provided in China | **China Satcom: transponder leas VSAT**, **broadcast TV**, **maritime/aviation**, **emergency communications**. **Tiantong satellite system mobile satellite voice and data** services across China. **China SatNet:** building a **LEO satellite Inter system** for broadband access.

List some content providers and describe their network architectures briefly | **Netflix / YouTu global CDN architecture**, **edge caching inside ISPs**, heavy use of **peering and IXPs**. **Tencent Video iQIYI: hierarchical CDN (core–regional–edge)**, **private backbone networks**, integrated with **cloud platforms**.

Some knowledge about peering. What is an IXP? How does an IXP earn money? | **Peering:** netw **exchange traffic directly** to reduce **cost and latency**. **IXP (Internet Exchange Point):** a **shared switching infrastructure** enabling multiple networks to peer. **IXP income:** mainly **port fees**, plus **membership fees and value-added services**.

IXPs distribution in the world. Please introduce some important IXPs | IXPs are **highly concent in Europe and North America**, with fast growth in **Asia and Africa**. **Important IXPs: DE-CIX (Frank AMS-IX (Amsterdam)**, **LINX (London)**, **IX.br (Brazil)**, **Equinix Internet Exchange**, **HKIX (Hong Kong**

Why is scheduling needed? | **Multiple flows share the same output link** | **Scheduling decides which packet is transmitted next on a link**

Basic scheduling criteria | **Throughput**: maximize link utilization | **Delay**: minimize average packet delay | **Fairness**: allocate bandwidth fairly among flows

FIFO scheduling (First-In First-Out) | **Rule**: packets are transmitted in the order they arrive | **Characteristics**: simple to implement | **Limitation**: no differentiation between flows, aggressive flows can dominate

Priority scheduling | **Rule**: packets are classified into priority classes | higher-priority packets are transmitted first | **Advantage**: supports delay-sensitive traffic (e.g., voice, video) | **Problem**: lower-priority traffic may starve

Round-robin scheduling | **Rule**: each flow is served in turn, one packet per round | **Advantage**: improves fairness compared to FIFO | **Limitation**: does not account for packet size differences

Weighted fair scheduling (concept) | **Rule**: each flow is assigned a weight | service rate proportional to weight | **Advantage**: flexible fairness and bandwidth guarantees | **Use case**: quality of service (QoS) support

三、Match + Action 抽象（Match + Action Abstraction)

. 核心思想

每个到达的数据包（incoming packet）：

- **Match（匹配）**：检查其协议头字段（header fields）是否符合 **流表**（flow table） 中的某条规则（rule）。
- **Action（动作）**：若匹配成功，执行 **预定义动作**（predefined action），如：
  - 转发到某端口（forward to port）、
  - 丢弃（drop）、
  - 修改 TTL（modify TTL）等。

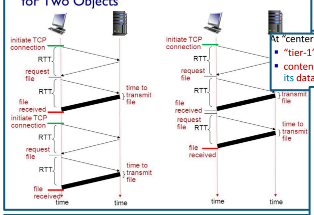
表（Flow Table） 包含：

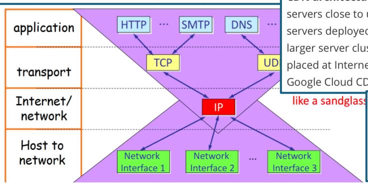
- **匹配字段**（Match fields）
- **优先级**（Priority，用于解决规则冲突 / resolve rule conflicts）
- **动作**（Action）
- **计数器**（Counters，统计匹配的数据包数 / #packets 和字节数 / #bytes）

Distance Vector algorithm (Bellman–Ford) | **Foundation**: Bellman–Ford equation $D_x(y) = \min_v\{c(x,v) + D_v(y)\}$ | **Local view**: each node knows costs to neighbors only | **Information exchange**: nodes periodically exchange distance vectors with neighbors | **Iteration**: iterative, asynchronous updates until convergence

Distance Vector algorithm features | **Distributed**: each node communicates only with directly connec neighbors | **Iterative**: updates continue until no changes occur | **Asynchronous**: nodes do not need to operate in lockstep | **Self-stopping**: no action once routing tables converge

Non-persistent HTTP v.s. Persistent HTTP for Two Objects





like a sandglass

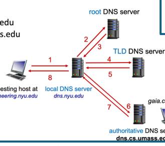
CDN architecture approaches | **Enter deep**: push CDN servers deep into many access networks | place servers close to users to reduce latency and backbone traffic | **Early Akamai example**: about 240,000 servers deployed in more than 120 countries (2015) | **Bring home**: deploy a smaller number (tens) of larger server clusters in POPs near access networks | **Deployment location**: CDN clusters are typically placed at Internet Exchange Points (IXPs) | **Major CDN providers**: Cloudflare | Amazon CloudFront | Google Cloud CDN

CDN operation (application layer) | **Content storage**: CDN nodes store copies series *MAD MEN* | **Request process**: subscriber requests content and the service provider returns a manifest file | **Content delivery**: client uses the manifest to retrieve content at the highest supported rate | **Adaptation mechanism**: client may switch to a different bitrate or a different content copy if the network path becomes congested

DNS name resolution: iterated query

Example: host at engineering.nyu.edu wants IP address for gaia.cs.umass.edu

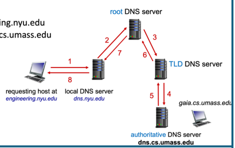
Iterated query:
- contacted server replies with name of server to contact
- "I don't know this name, but ask this server"



DNS name resolution: recursive query

Example: host at engineering.nyu.edu wants IP address for gaia.cs.umass.edu

Recursive query:
- puts burden of name resolution on contacted name server
- heavy load at upper levels of hierarchy?



Differences among HTTP/1.0, 1.1, 2.0, 3.0 | **HTTP/1.0:** non-persistent connections, **one request per TCP**, high overhead. **HTTP/1.1: persistent connections**, pipelining, chunked transfer, better efficiency. **HTTP/2: binary framing**, **multiplexing**, header compression (HPACK), one TCP connection, lower latency. **HTTP/3:** runs over **QUIC (UDP)**, avoids TCP HOL blocking, **faster connection setup**, better performance on lossy networks.

Modern E-mail transfer system: Does SMTP use intermediate mail servers? | **Yes.** SMTP usually uses **multiple intermediate mail servers (MTAs)**; mail is **store-and-forwarded** hop by hop until reaching the recipient's mail server.

DNS server behaviors (root, TLD, authoritative, local) | **Root/TLD/Authoritative servers:** respond **iteratively** (return referrals). **Local DNS server:** performs **recursive queries** on behalf of clients.

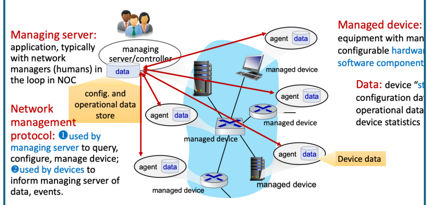
Are DNS queries carried over UDP or TCP? | **Both. UDP** is used by default (fast, low overhead). **TCP** is used for **large responses**, **zone transfers**, or when UDP responses are truncated.

CDN server placement philosophies | **Enter Deep:** deploy servers **inside access ISPs**, very close to users, **low latency**, high deployment cost (**larger market share, e.g., Akamai**). **Bring Home:** place servers at **centralized IXPs/data centers**, easier management, slightly higher latency.

Principle of DASH | **DASH (Dynamic Adaptive Streaming over HTTP):** video encoded at **multiple bitrates**, client **dynamically selects bitrate** per segment based on bandwidth and buffer status, improving QoE.

Socket programming with TCP | **TCP sockets:** connection-oriented, reliable byte stream. **Server:** socket → bind → listen → accept → send/recv. **Client:** socket → connect → send/recv. Ensures **reliable, ordered delivery**.

Components of network management


managing server/controller
data
config. and operational data store
agent
data
managed device
Device data

Managing server: application, typically with network managers (humans) in the loop in NOC

Network management protocol: ❶used by managing server to query, configure, manage device; ❷used by devices to inform managing server of data, events.

Managed device: equipment with man configurable hardwa software component

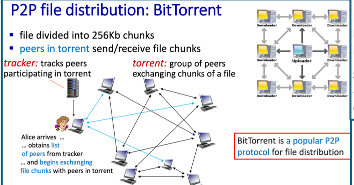
Data: device "st configuration da operational data device statistics

P2P file distribution: BitTorrent

- file divided into 256Kb chunks
- peers in torrent send/receive file chunks

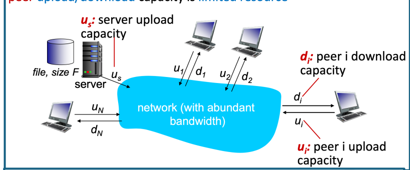
tracker: tracks peers participating in torrent

torrent: group of peers exchanging chunks of a file


Alice arrives …
… obtains list of peers from tracker
… and begins exchanging file chunks with peers in torrent
Downloader
Uploader

BitTorrent is a popular P2P protocol for file distribution

peer upload/download capacity is limited resource


u_s: server upload capacity
d_i: peer i download capacity
u_i: peer i upload capacity
file, size F
u_s
server
network (with abundant bandwidth)
u_1
d_1
u_2
d_2
u_N
d_N
d_i
u_i

1. **Is TCP a GBN or an SR protocol?** | TCP is **closer to SR (Selective Repeat)**: the receiver **buffers out-of-order segments** and sends **cumulative ACKs** (with SACK option), unlike pure GBN which discards out-of-order packets.

2. **Compare flow control and congestion control** | **Flow control:** prevents **sender overwhelming receiver**, based on **receiver window (rwnd)**. **Congestion control:** prevents **overloading the network**, based on **congestion window (cwnd)** and network feedback (loss/RTT).

3. **TCP global synchronization phenomenon** | Many TCP flows **detect congestion simultaneously**, then **reduce cwnd at the same time**, causing **periodic link underutilization and oscillation**.

4. **TCP congestion control details** | Includes **Slow Start** (cwnd exponential growth), **Congestion Avoidance** (linear increase), **Fast Retransmit** (on 3 duplicate ACKs), and **Fast Recovery** (avoid returning to slow start after mild loss).

5. **Fairness problem of TCP** | TCP aims for **rough bandwidth fairness**, but flows with **shorter RTTs** or **more aggressive variants** get more bandwidth; non-TCP-friendly flows can be **unfair**.

6. **Evolving transport-layer functionality (HTTP/3)** | **HTTP/3 runs over QUIC (UDP)**, integrates **transport + security**, supports **stream-level multiplexing**, avoids **TCP head-of-line blocking**, and enables **faster connection setup**.

7. **What is SYN flooding? Possible solutions?** | **SYN flooding:** attacker sends大量 **SYN packets** without completing handshake, exhausting server resources. **Solutions: SYN cookies**, **rate limiting**, **firewalls**, and **increasing backlog queues**.

Describe how packet loss can occur at input ports & how to eliminate it | **Input-port loss:** packets arrive faster than they can be transferred through the **switch fabric**, causing **input buffer overflow**. **Elimination (without infinite buffers):** use **faster switch fabric**, **input scheduling**, **virtual output queues (VOQ)**, and **backpressure/flow control**.

Describe how packet loss can occur at output ports & can higher fabric speed prevent it? | **Output-port loss:** multiple inputs send packets to the **same output port simultaneously**, exceeding output buffe capacity. **Higher fabric speed:** helps but **cannot fully prevent loss** if the **output link rate** is the bottleneck.

What is HOL blocking? Where does it occur? | **HOL (Head-of-Line) blocking:** a packet at the **front of an input queue** blocks packets behind it destined for free outputs. It occurs at **input ports** (with FIFO queues).

Why is IPv6 adoption so slow? | **IPv4 NAT extends lifetime**, **high migration cost**, **legacy equipment/software**, **limited immediate economic incentive**, and **need for dual-stack operation**.

Compare IPv4 and IPv6 headers & benefits of IPv6 | **IPv4:** variable-length header, **checksum**, **fragmentation by routers**, 32-bit addresses. **IPv6: fixed 40-byte header**, **no header checksum**, **no router fragmentation**, 128-bit addresses. **Benefits: larger address space**, **faster forwarding**, **simpler header**, **better support for mobility and security (IPsec)**.

Difference among Firewalls, IDS, and IPS | **Firewall: blocks/allows traffic** based on rules. **IDS: detects and alerts** on attacks (passive). **IPS: detects and actively blocks** malicious traffic (inline).

IPv6 over IPv4 tunneling as link-layer protocol — agree? | **Yes (conceptually).** IPv6 is **encapsulated inside IPv4**, and IPv6 treats the tunnel as a **virtual link**, similar to a **link-layer**, hiding the underlying IPv4 network.

Compare the mechanisms of CSMA/CD and CSMA/CA | **CSMA/CD** (Ethernet): detect collisions while **transmitting**, then **abort + binary exponential backoff**; requires **wired medium** where collision detection is feasible. **CSMA/CA (802.11): collision avoidance** using **random backoff**, **RTS/CTS**, and ACKs; designed for **wireless** where collision detection is hard.

Compare the frame structures of Ethernet and 802.11 | **Ethernet frame:** simple header (**Dst MAC, Src MAC, Type/Length**), payload, **CRC**, no ACK field. **802.11 frame:** more complex, **multiple address fields (up to 4)**, **control fields**, **sequence numbers**, supports **ACK/RTS/CTS**, higher overhead.

Differences between a Bluetooth master and an 802.11 base station (AP) | **Bluetooth master: controls timing and polling**, slaves transmit only when permitted, forms a **piconet**. **802.11 AP: does not centrally schedule transmissions**, stations contend via CSMA/CA, AP mainly provides **association and bridging**.

Why are ACKs used in 802.11 but not in wired Ethernet? | **Wireless links are error-prone** (noise, fading, hidden terminals), so **ACKs ensure reliability**. **Wired Ethernet** has **low bit error rate** and relies on **upper layers (TCP)** for reliability, avoiding extra overhead.

If RTS/CTS frames were as long as DATA/ACK, would there be any advantage? | **No.** RTS/CTS reduce collisions by sending **short control frames**. If they were as long as DATA/ACK, **overhead would negate benefits**, making performance worse.

AI for networking: directions and applications | **Directions: traffic prediction**, **congestion control**, **routing optimization**, **network automation (self-driving networks)**. **Applications: anomaly/intrusion detection**, **QoS optimization**, **data center traffic engineering**, **wireless resource management**.

1. **Brief introduction of 802.1, 802.3, 802.11** | **IEEE 802.1: LAN bridging & control** (e.g., **VLAN 802.1Q**, **STP/RSTP**, link aggregation). **IEEE 802.3: Ethernet (wired)** MAC/PHY standards (frame format, speeds). **IEEE 802.11: Wi-Fi (wireless LAN)** MAC/PHY (CSMA/CA, association, multiple PHY generations).

2. **Trends in data center networking (literature + textbook ideas)** | **Clos/leaf–spine becomes default** for east–west traffic + **ECMP** scalability (merchant silicon). (Google Research) **L3 underlay + VXLAN/EVPN overlays** to avoid L2 STP limits, support multi-tenancy and scalable L2/L3 virtualization. (Juniper Networks) **More programmability/automation:** SDN control + telemetry; **open networking NOS (e.g., SONiC)** to decouple hardware/software. (Sonic Foundation) **AI/ML-era fabrics:** higher bisection bandwidth and loss/latency sensitivity → more focus on congestion control and high-speed optics; optical switching is explored/used at scale in some hyperscalers. (Stanford University)

3. **Introduction of L2/L3/L4 switches** | **L2 switch:** forwards by **MAC table**, same broadcast domain, supports VLAN. **L3 switch:** does **IP routing** (inter-VLAN), uses routing table/ASIC for high-speed forwarding. **L4 switch:** traffic steering by **TCP/UDP ports** (often as **load balancer**: 5-tuple hashing, NAT, health checks).

4. **Explanation about CRC** | **CRC (Cyclic Redundancy Check):** sender computes a **checksum via polynomial division** over bits; receiver recomputes—if mismatch → **detect error** (strong burst-error detection), but **does not correct** errors.

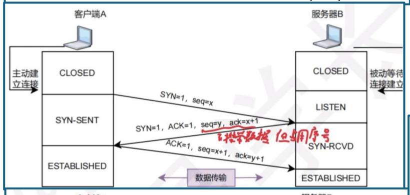
5. **Binary exponential backoff algorithm** | After a collision/failure, choose random (k\in[0,2^i-1]) slots to wait, where **i increases after each collision** (up to a max), so contention reduces exponentially (classic in **Ethernet CSMA/CD**).

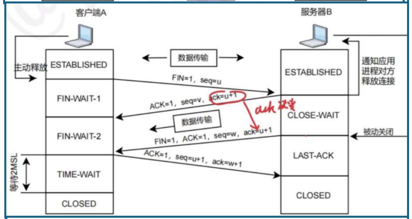
6. **Link-layer services to network layer; which exist in IP? in TCP?** | Possible L2 services: **framing**, **link access (MAC)**, **error detection** (CRC), **reliable delivery** (retransmissions, e.g., Wi-Fi ARQ), **flow control** (pause/credit), sometimes **link-layer security**. **In IP:** mainly **best-effort forwarding** (no reliability/flow control), limited **error reporting via ICMP**. **In TCP: reliable delivery**, **flow control**, **congestion control** (end-to-end).

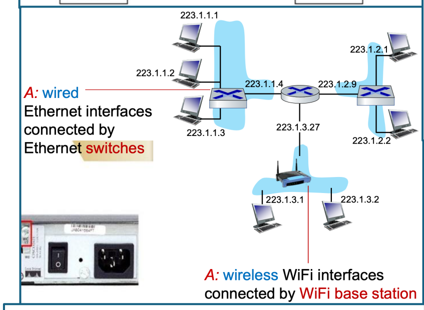
7. **L2 execution: hardware vs software, and interaction** | **Fast path in hardware (NIC + switch ASIC):** MAC forwarding, VLAN tags, CRC check, queueing/scheduling. **Control/management in software (driver/OS/NOS):** config tables (MAC/VLAN/ACL), handles exceptions, updates rules; hardware applies these rules at line rate while software **programs and monitors** the hardware.

8. **Compare flow control in data link layer and transport layer** | **L2 flow control: hop-by-hop**, protects **one link/next hop** (e.g., PAUSE/credits), can cause head-of-line effects across a path. **Transport (TCP) flow control: end-to-end**, receiver advertises **rwnd** so sender won't overwhelm the receiver buffer (independent of network congestion control via cwnd).

Content Distribution Networks (CDNs) | **Core challenge**: how to stream content selected from millions of videos to hundreds of thousands of simultaneous users | **Basic idea (Option 2)**: store and serve multiple copies of videos at geographically distributed sites (CDN)

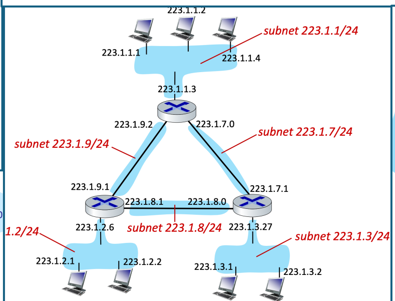


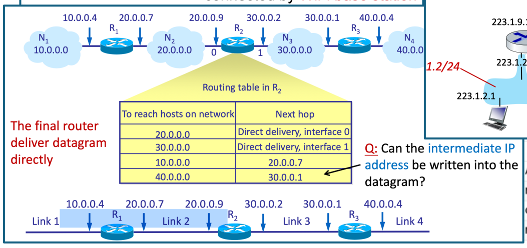




A: wired Ethernet interfaces connected by Ethernet switches

A: wireless WiFi interfaces connected by WiFi base station



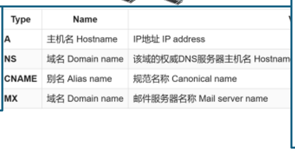


Routing table in R_2

To reach hosts on network	Next hop
20.0.0.0	Direct delivery, interface 0
30.0.0.0	Direct delivery, interface 1
10.0.0.0	20.0.0.7
40.0.0.0	30.0.0.1

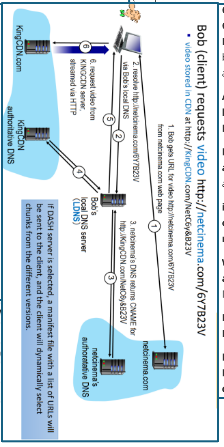
The final router deliver datagram directly

Q: Can the intermediate IP address be written into the datagram?



Type	Name	
A	主机名 Hostname	IP地址 IP address
NS	域名 Domain name	该域的权威DNS服务器主机名 Hostnam
CNAME	别名 Alias name	规范名称 Canonical name
MX	域名 Domain name	邮件服务器名称 Mail server name

Bob (client) requests video http://netcinema.com/6Y7B23V
- video stored in CDN at http://KingCDN.com/NetC6y&B23V


1. Bob gets URL for video http://netcinema.com/6Y7B23V from netcinema.com web page
1
netcinema.com
2. resolve http://netcinema.com/6Y7B23V via Bob's local DNS
2
5
3. netcinema's DNS returns CNAME for http://KingCDN.com/NetC6y&B23V
3
netcinema's authoratative DNS
Bob's local DNS server (LDNS)
6. request video from KINGCDN server, streamed via HTTP
6
4
KingCDN.com
KingCDN authoritative DNS

If DASH server is selected, a manifest file with a list of URLs will be sent to the client, and the client will dynamically select chunks from the different versions.

Compare and contrast link-state vs distance-vector; static vs dynamic routing | **Link-state (e.g., OSPF):** routers flood **full topology**, compute routes with **Dijkstra**, **fast convergence**, higher overhead. **Distance-vector (e.g., RIP):** routers exchange **distance vectors** with neighbors, **Bellman–Ford**, simpler but **slow convergence** and **count-to-infinity**. **Static routing:** manually configured, **no overhead**, not adaptive. **Dynamic routing:** automatically adapts to topology changes, **scalable but complex**.

Are RIP/OSPF/BGP load-insensitive? True or False? How to solve the problem? | **True.** They mainly use **hop count, link cost, policy**, not real-time load, causing **congestion**. **Solutions: traffic engineering**, **ECMP**, **MPLS**, **SDN-based routing**, or **load-aware metrics**.

Example of BGP incidents and reasons | **2008 Pakistan–YouTube incident:** Pakistan ISP **mis-announced a more specific prefix**, hijacking YouTube traffic globally. **Reason: lack of prefix validation** and **trust-based BGP design**.

iBGP split horizon rule: basics and necessity | **Rule:** routes learned via **iBGP are not advertised to other iBGP peers**. **Necessity:** prevents **routing loops** inside an AS; requires **full-mesh iBGP** or **route reflectors**.

Why different inter-AS and intra-AS routing protocols? | **Intra-AS (OSPF/RIP):** focus on **performance and fast convergence**. **Inter-AS (BGP):** focus on **policy control, scalability, and business relationships**. Different goals require different designs.

Four types of ICMP messages | **Echo request/reply**, **Destination unreachable**, **Time exceeded**, **Redirect**.

BGP and IP-anycast (used in DNS) | **IP-anycast:** multiple servers share **same IP prefix**, announced via BGP. **Traffic goes to nearest server (BGP-wise)**. **Used in DNS root/TLD servers** for **low latency, load distribution, and fault tolerance**.